

ACM India Summer School on Symmetric Key Cryptography

Day 3: Implementation of Block Ciphers (AES-128 and DES)

Sourav Sharma

June 10, 2026

1 Introduction

Block ciphers are a cornerstone of symmetric key cryptography. In this lab, I implemented the complete encryption and decryption pipelines for two major block ciphers: the Data Encryption Standard (DES) and the Advanced Encryption Standard (AES-128). Both implementations were written in clean, comment-free C++ (using a Competitive Programming style) and verified using standard test vectors.

2 Data Encryption Standard (DES)

DES is a symmetric block cipher that operates on 64-bit blocks of data using a 56-bit key. It relies on a Feistel network structure consisting of 16 rounds of substitution and permutation.

2.1 Key Schedule

The 64-bit master key K contains 8 parity bits, which are discarded. The remaining 56 bits are permuted and split into two 28-bit registers, C_0 and D_0 , using the Permuted Choice 1 (PC-1) table. In each round $i \in \{1, \dots, 16\}$, the registers are left circularly shifted according to a predefined shift schedule (either 1 or 2 positions):

$$C_i = \text{LS}_i(C_{i-1}), \quad D_i = \text{LS}_i(D_{i-1})$$

The concatenated 56-bit state $C_i \parallel D_i$ is then compressed to a 48-bit round subkey K_i using the Permuted Choice 2 (PC-2) table.

2.2 Feistel Round Function

The Feistel function $f(R, K_i)$ takes the 32-bit right half of the state R and the 48-bit subkey K_i :

1. **Expansion (E):** The 32-bit block is expanded to 48 bits using the expansion table E , duplicating certain bits.
2. **Key Addition:** The expanded block is XORed with the 48-bit subkey: $X = E(R) \oplus K_i$.
3. **Substitution (S -boxes):** The 48-bit result is split into eight 6-bit groups. For each group $j \in \{1, \dots, 8\}$, the outer bits determine the row, and the inner bits determine the column of the corresponding S -box S_j . Each S -box maps 6 bits to 4 bits.
4. **Permutation (P):** The concatenated 32-bit output of the S -boxes is permuted using the table P .

2.3 Encryption and Decryption Structure

The input block is first permuted via the Initial Permutation (IP). The state is split into L_0 and R_0 . For $i = 1$ to 16, the round updates are:

$$L_i = R_{i-1}, \quad R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$$

After the 16th round, the final state is swapped ($R_{16} \parallel L_{16}$) and permuted using the Final Permutation (FP), which is the inverse of IP. Decryption uses the exact same structure but applies the subkeys in reverse order ($K_{16}, K_{15}, \dots, K_1$).

2.4 Verification

I compiled and ran `DES_Solution.cpp` against `DES_TestVectors.csv`. The implementation successfully passed all **100/100 test vectors** (containing weak keys, semi-weak keys, and random inputs).

3 Advanced Encryption Standard (AES-128)

AES-128 is a SP-network block cipher that processes 128-bit blocks using a 128-bit key. The block state is formatted as a 4×4 byte grid (column-major order).

3.1 Key Schedule

The 128-bit master key is expanded into 44 words ($w_0, \dots, w_{43}$) of 32 bits each, providing 11 round keys (each of 128 bits). For indices not divisible by 4, $w_i = w_{i-4} \oplus w_{i-1}$. For i divisible by 4, a non-linear transformation is applied to w_{i-1} before XORing:

$$\text{SubWord}(\text{RotWord}(w_{i-1})) \oplus \text{Rcon}[i/4]$$

where `RotWord` performs a 1-byte left circular shift, `SubWord` applies the AES S-box to each of the 4 bytes, and `Rcon` contains the round constants in $\text{GF}(2^8)$.

3.2 Round Operations

AES-128 performs 10 rounds of four mathematical operations:

1. **SubBytes**: A non-linear substitution step where each byte is replaced with its sub-byte using the AES S-box (constructed from the multiplicative inverse over $\text{GF}(2^8)$ followed by an affine transformation).
2. **ShiftRows**: A transposition step where the rows of the state grid are cyclically shifted left by offset $r \in \{0, 1, 2, 3\}$.
3. **MixColumns**: A linear transformation step where each column of the state is multiplied by a fixed polynomial over $\text{GF}(2^8)$ modulo $x^4 + 1$. The multiplication by 2 is implemented using a left shift and a conditional XOR with `0x1b` (the reduction polynomial for $\text{GF}(2^8)$: $x^8 + x^4 + x^3 + x + 1$).
4. **AddRoundKey**: The state bytes are XORed with the corresponding round key bytes.

In the final round (Round 10), the `MixColumns` step is omitted. Decryption is performed by applying the inverse operations (`InvSubBytes`, `InvShiftRows`, `InvMixColumns`) with the round keys in reverse order.

3.3 Verification

I compiled and tested `AES_Solution.cpp` against `AES_TestVectors.csv`. The implementation successfully passed all **100/100 test vectors**.